UFMG
UNIVERSIDADE FEDERAL DE MINAS GERAIS

Programação e Desenvolvimento de Software 2

Tipos Abstratos de Dados (TADs)

Prof. Douglas G. Macharet
douglas.macharet@dcc.ufmg.br

DCC
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

1

Introdução

- Algoritmo
  - Sequência de ações executáveis
  - Transformam uma entrada em uma saída
  - Trabalham sobre estruturas de dados
- Estruturas de dados
  - Informações que representam uma situação real
  - Abstração da realidade
  - Suportam as operações dos algoritmos

2

Introdução

- Diferença entre Programa e Algoritmo?
  - Um programa é uma realização concreta de um algoritmo abstrato, baseado em representações de dados específicas
  - Programas precisam ser implementados numa linguagem que pode ser entendida e seguida pelo computador
    - C, C++, Java, Python, ...

Introdução

- Como declarar variáveis no programa?
- O que é um tipo de dado?
- O que os tipos "int" e "bool" representam?
  - Em quais operações podem ser utilizados?
- Programa possuirá apenas tipos como esses?
  - Primitivos
  - Como criar conceitos mais genéricos?

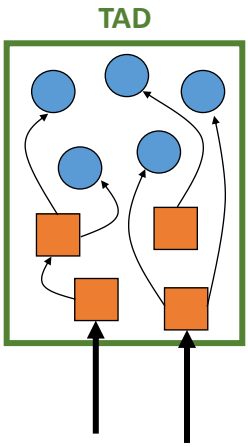
Definir nossos próprios tipos!

Introdução

- Quando um programa é considerado "bom"?
  1. Funciona (faz o que foi especificado)
  2. Fácil de entender e modificar
  3. Razoavelmente eficiente (recursos)
- Bons programas fazem uso de Abstração
  - Conceito (ideia) → Implementação (concreto)
  - Identificação e definição dos tipos específicos
- Tipos Abstratos de Dados
  - Ajudam fazer (2), que facilita alcançar (1)

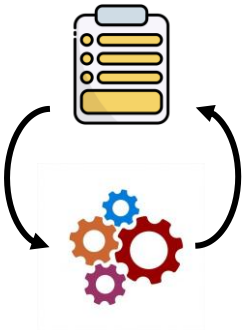
Tipos Abstratos de Dados (TADs)

- Generalização de tipos primitivos
- **Encapsula** diferentes elementos (membros)
  - Conjunto de **valores** (atributos)
  - Conjunto de **operações** (métodos)
- Código com perspectivas diferentes
  - Usuário do TAD x Programador do TAD
  - Acesso via as operações disponibilizadas
  - Conhecimento/definição da estrutura interna

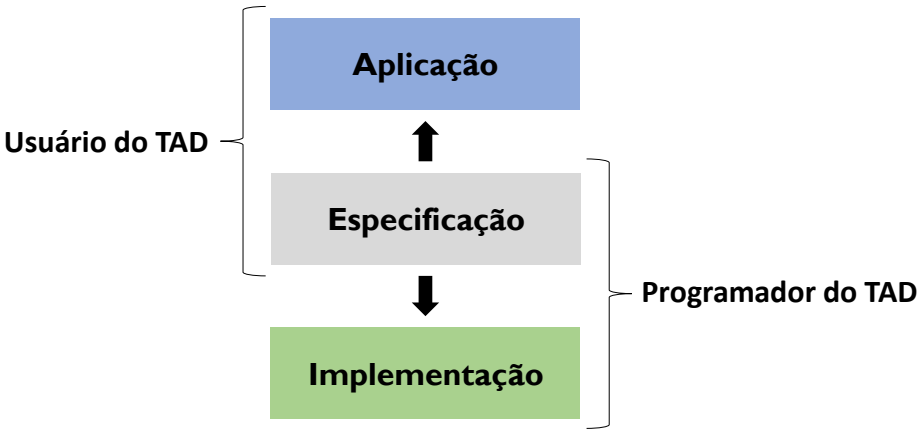


Tipos Abstratos de Dados (TADs)

- Desvincular a <u>especificação</u> da sua <u>implementação</u>
  - Ocultação de informação (*information hiding*)
- **Especificação** (interface, contrato)
  - <u>O que</u> esse tipo de "coisa" representa?
  - <u>Quais</u> operações podem ser feitas com ela?
- **Implementação** (código)
  - <u>Como</u> essa "coisa" deve ser implementada?
  - <u>Como</u> as operações funcionam internamente?



Tipos Abstratos de Dados (TADs)



Tipos Abstratos de Dados (TADs)

- Um TAD é um tipo caracterizado <u>não pelos valores</u> que o compõe, mas <u>pelas operações</u> que se aplicam sobre ele

> O que o TAD **representa e faz** é mais importante do que **como** ele faz!

Tipos Abstratos de Dados (TADs)

- Lista de números: 4, 1, 5, 6, ...
  - Quais operações podemos ter?
    - Criar uma nova lista, inserir um número, remover, ordenar, ...
  - Como deve ser feita a implementação?

Usuário do TAD

```
int main() {
  Lista *l;
  int x = 20;

  l->inserir(x);
  ...
}
```

Programador do TAD

- Vetor: 20 | 13 | 02 | 30

```
void inserir(int x) {
  l->arranjo[...] = x;
  ...
}
```

- Lista: → 20 → 13 → 02 → 30 →

```
void inserir(int x) {
  Celula *c = criar_celula(x);
  l->ultimo = c;
}
```

Tipos Abstratos de Dados (TADs)

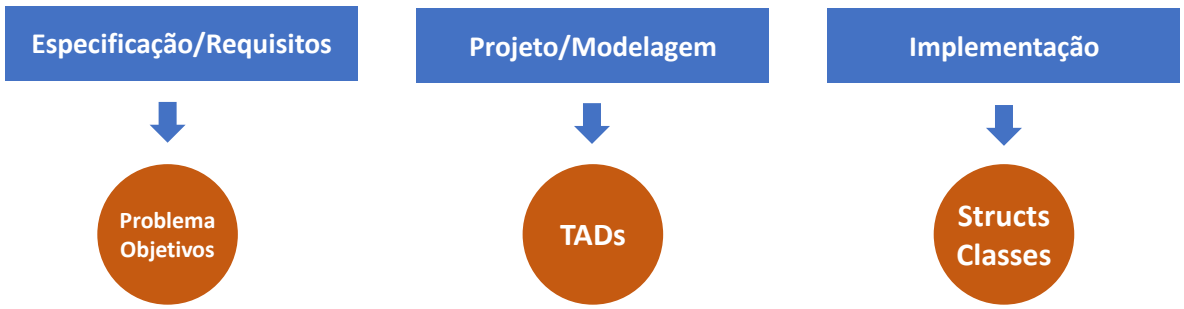
- Quais as vantagens disso?
  - Integridade, manutenção, reutilização, ...
  - Podemos modificar a implementação interna do TAD sem que o usuário do TAD também precise fazer alguma alteração!
- Exemplos
  - Personagem, Aluno, Carro
  - Lista, Árvore, Pilha, Fila
  - Quais operações são aplicáveis nesses casos?

Tipos Abstratos de Dados (TADs)
Operações

- Quais operações devem representar um TAD?
- Filosofia egoísta
  - Se algo não é útil, então não é necessário
  - Informe ao usuário apenas o que ele precisa
    - Lembre-se do princípio anterior
    - Desejo x Necessidade
    - Liberdade para alterações futuras

Tipos Abstratos de Dados (TADs)

Níveis de detalhamento



Implementação de TADs

- Em linguagens estruturadas (C, Pascal), a implementação é feita com definições de tipos e implementação de funções
  - *typedef*
- Em linguagens orientadas a objetos a implementação é feita através de classes (começaremos usando structs!)
  - C++, Java, C#, ...

Structs

- Em **C++** uma **struct** é <u>similar</u> a uma **classe**
  - Diferença no nível de proteção (outra aula!)
- Conjunto de variáveis (possivelmente de tipos diferentes) e métodos juntos/acessíveis sob um único nome (significado)
- Ponto 3D
  - x, y, z
  - Ao invés de criar três variáveis, é possível criar uma única estrutura (tipo) contendo os três campos (mais representativo)

Structs
Exemplo 1

Funções matemáticas

```
#include <iostream>
#include <cmath>

using namespace std;

struct Ponto3D {
  double x;
  double y;
  double z;

  double calcularDistancia(Ponto3D* p2) {
    double dx = p2->x - this->x;
    double dy = p2->y - this->y;
    double dz = p2->z - this->z;
    return sqrt(dx*dx + dy*dy + dz*dz);
  }
};
```

Dados/Atributos

Operação

<u>Ponteiro</u> para a **própria** estrutura. Não é obrigatório, mas facilita o entendimento. Será melhor discutido no futuro.

Manipulação dos dados

Atenção ao 'ponto e vírgula' na definição!

https://www.cplusplus.com/reference/cmath/

Structs
Exemplo 1

Acessando os dados.

```
int main() {

  Ponto3D* p1 = new Ponto3D();
  p1->x = 0.0;
  p1->y = 0.0;
  p1->z = 0.0;

  Ponto3D* p2 = new Ponto3D();
  p2->x = 5.0;
  p2->y = 5.0;
  p2->z = 5.0;

  cout << p1->calcularDistancia(p2) << endl;

  delete p1;
  delete p2;

  return 0;
}
```

Alocando as diferentes variáveis na memória.

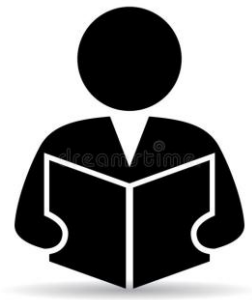
Chamando um método.

Atenção!

Pythontutor Wandbox

Structs
Exemplo 2

- Como representar um Aluno?
  - Quais atributos/dados?
    - Nome, matrícula, curso, ...
  - Quais operações sobre esses dados?
    - Matricular, Calcular RSG, ...



Structs
Exemplo 2

```
#include <iostream>
#include <string>

using namespace std;

struct Aluno {
  string nome;
  int matricula;

  float calcularRSG() {
    // Fazer a conta necessaria
    return 0;
  }
};

int main() {
  Aluno aluno;
  aluno.nome = "Jose da Silva";
  aluno.matricula = 201812345;
  cout << aluno.nome << endl;
  return 0;
}
```

Essa variável está na **PILHA**, por isso não preciso do delete.

Pythontutor Wandbox

Structs
Exemplo 3

```
#include <iostream>

// Declaração do TAD
struct MyTAD {

    int x = 0;

    void set_x(int x) {
        this->x = x;
    }

    void who_am_i() {
        std::cout << this << std::endl;
    }

};
```

Usamos o ponteiro 'this' para acessar os dados da estrutura.

O 'this' é um *auto-ponteiro*, logo, será impresso o próprio endereço.

Structs
Exemplo 3

```
int main() {

    MyTAD tad_stack;                // Referência, STACK

    std::cout << &tad_stack << std::endl;
    tad_stack.who_am_i();
    tad_stack.set_x(55);
    tad_stack.x = 99;

    MyTAD *tad_heap = new MyTAD(); // Ponteiro, HEAP

    std::cout << tad_heap << std::endl;
    tad_heap->who_am_i();

    delete tad_heap;

    MyTAD outra_var_stack;
    outra_var_stack = tad_stack;

    MyTAD *outra_var_heap;
    outra_var_heap = &tad_stack;

    return 0;
}
```

Se for **referência**, usa-se o operador '.'

Se for **ponteiro**, usa-se o operador '->'

O 'this' dentro de MyTAD é um ponteiro para o próprio endereço. Logo, essas linhas devem imprimir a mesma coisa!

O acesso dos dados pode ser feita via método (recomendado) ou diretamente.

Ao se fazer isso ocorre uma cópia dos valores!

Simplesmente aponta para o endereço.

Pythontutor

Structs
Construtores

- Tipo especial de membro do TAD
  - Implícitos (default) / Explícitos
- Ajudam na inicialização dos atributos
- São chamados na declaração/alocação

```
struct Ponto3D {
  double x;
  double y;
  double z;

  Ponto3D() {
    this->x = -1.0;
    this->y = -1.0;
    this->z = -1.0;
  }
};
```

Não é necessário, mas deixa claro que está chamando uma função!

```
int main() {

  Ponto3D* p = new Ponto3D();
  p->x = -1.0;
  p->y = -1.0;
  p->z = -1.0;

  return 0;
}
```

Structs
Construtores

- Sobrecarga: métodos com mesmo nome
  - Facilitam e deixam o código mais limpo
  - Estudaremos com mais detalhes!

```
int main() {

  Ponto3D* p = new Ponto3D();

  Ponto3D* p = new Ponto3D(1.0, 5.0, 3.0);

  return 0;
}
```

Wandbox

```
struct Ponto3D {
  double x;
  double y;
  double z;

  Ponto3D() {
    this->x = -1.0;
    this->y = -1.0;
    this->z = -1.0;
  }

  Ponto3D(double v) {
    this->x = v;
    this->y = v;
    this->z = v;
  }

 Ponto3D(double x, double y, double z) {
    this->x = x;
    this->y = y;
    this->z = z;
  }
};
```


Desenvolvimento

- Desenvolvimento Orientado a Objetos
  - Construção de programas como uma coleção estruturada de implementações completas/parciais de TADs (structs/classes)
- Modularização
  - Programação (design) por contrato
  - Programação Orientada a Objetos

Desenvolvimento
Contrato

- O que é um contrato?
  - Acordo entre duas ou mais partes
  - Todas as partes envolvidas devem cumprí-lo
- Quais contratos existem em um programa?
  - Requisitos de funcionalidade, desempenho, ...
  - Operações disponíveis em um TAD
    - Entradas e saídas

Desenvolvimento
Contrato

- Como podemos garantir (forçar) que o contrato definido estará sendo cumprido pelo Usuário do TAD?
  - Tipos
  - Asserções
  - Exceções
- Requisitos ←→ Implementação
  - Garantir a consistência entre esses artefatos

Desenvolvimento

- Qual implementação é melhor?

```
struct TriangRetangulo {
  float base, altura;
}
```

```
struct TriangRetangulo {
  float base, hipot, ang;
}
```

- A implementação não é o mais importante
  - Pense no tipo como um conjunto de operações
    - Criar, SetarBase, SetarAltura, CalcularArea, ...
  - Usuário deve acessar os dados pelas operações!

Desenvolvimento

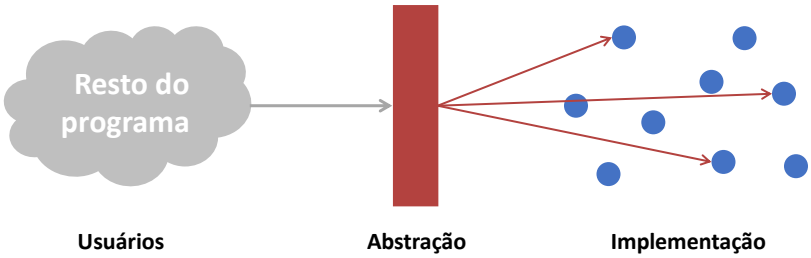
- Esses TADs são iguais ou diferentes?

```
struct Ponto2D {
  float x;
  float y;
}
```

```
struct Ponto2D {
  float r;
  float theta;
}
```

- Iguais (conceito) e Diferentes (dados/implementação)!
- Tipo/TAD
  - Elementos com propriedades semelhantes
  - Usuários dependem do conceito geral

Considerações finais



- Implementação não deve ser conhecida
  - A abstração deve definir o contrato (prover os métodos) que será utilizado para a manipulação dos dados (internos ao TAD)